

CS403 SOFTWARE ENGINEERING

Complete Course Notes with Coloured Vector Flowcharts

Definitions + Flowcharts + Comparison Tables + Editor-style Code + Real-World Links | 5 Units

SYLLABUS COVERED

Unit 1 The Software Product (characteristics, SW vs HW); The Software Process & Generic model; Process models: Waterfall, Prototype, RAD, Incremental, Spiral, Component-assembly, RUP, Agile; Process customization/improvement, CMM; Product & Process metrics	Unit 2 Requirement types (functional / non-functional, FURPS+); Sources & elicitation (interviews, questionnaires, brainstorming, JAD, prototyping, observation); Analysis modelling; Use cases; SRS (IEEE 830) with quality attributes; Validation, verification & traceability (RTM)	Unit 3 Design process & principles (cohesion/coupling); UML overview; Architectural views (4+1) & styles (data-centred, data-flow, layered, OO, client-server, MVC); UI design golden rules; Function-oriented design SA/SD with DFD & data dictionary; Component-based design; design metrics	Unit 4 Static vs dynamic analysis; Fagan inspections; Test process, V-Model & levels of testing; Black-box (ECP, BVA, decision table, state transition, error guessing) & White-box (coverage, cyclomatic); JUnit; integration strategies; system & specialised testing; test plan/thresholds, metrics & tools; OO vs structured testing
Unit 5 Maintenance (types, factors, legacy); SCM & version control (CIs, baselines, change control, git); re-engineering; PM (scope, feasibility, WBS); estimation (LOC, FP, COCOMO); scheduling (Gantt, PERT/CPM, critical path); risk assessment & mitigation; SQA	Style Every major diagram is a COLOURED VECTOR FLOWCHART (drawn, not ASCII) with a caption	Language note All code snippets Java / shell-commands in editor-style coloured boxes	How to use Read units in order; copy each flowchart on paper; re-derive the tables

Prepared as a self-study & exam-prep guide following the CS403 syllabus. All flowcharts are vector graphics so they stay crisp and colourful on screen and in print.

TABLE OF CONTENTS

The Software Product, Process & Process Models	software Component
Requirement Elicitation, Analysis & Specification	function verification
Software Design, UML, Architecture & UI Design	design component
Analysis & Software Testing	static v system/sp
Software Maintenance, SCM & Project Measurement	mainten Gantt · PE

Study tip: redraw each coloured flowchart on paper, then fill in the definition boxes and tables from memory. Finish every unit's Exercise line in writing.

UNIT 1 | THE SOFTWARE PRODUCT, PROCESS & PROCESS MODELS

1.1 The Software Product

WHAT IS SOFTWARE?

Software = a set of related computer programs, their documentation, data and operating procedures which together enable a machine to perform a desired task. It is a LOGICAL product - no mass production, no wear and tear, but it ages. Uniquely: it is engineered, not manufactured.

Characteristic	Meaning	Why it matters
Correctness	meets requirements & specs	wrong outputs are useless
Reliability	functions correctly over time	banking/medical safety
Maintainability	easy to change & fix	60-70% of cost is maintenance
Usability	easy to learn & use	user adoption & training cost
Efficiency	good use of time & memory	speed of response
Portability	moves to new environment	reuse across platforms
Reusability	components usable again	faster new products
Security	protects data from misuse	trust, legal compliance

Software vs hardware (they age & behave differently)

Point	Hardware	Software
Failure mode	wears out (physical)	does not wear; DETERIORATES via changes
Manufacturing	mass produced after design	developed/engineered once, copied free
Change	replace physical part	recode + retest
Cost driver	production materials	design & development effort
Failure cause	environment, usage	errors in requirements / design / code

1.2 The Software Process & its framework

SOFTWARE PROCESS

A sequence of activities, methods, practices and transformations used to develop and maintain software. A framework of 5 umbrella activities: COMMUNICATION (talk with customer), PLANNING (map the road), MODELING (create sketches - analysis & design models), CONSTRUCTION (code + test), DEPLOYMENT (deliver + feedback). Overlaid by umbrella tasks: quality assurance, change control, risk management, measurement.

```
... process.txt
```

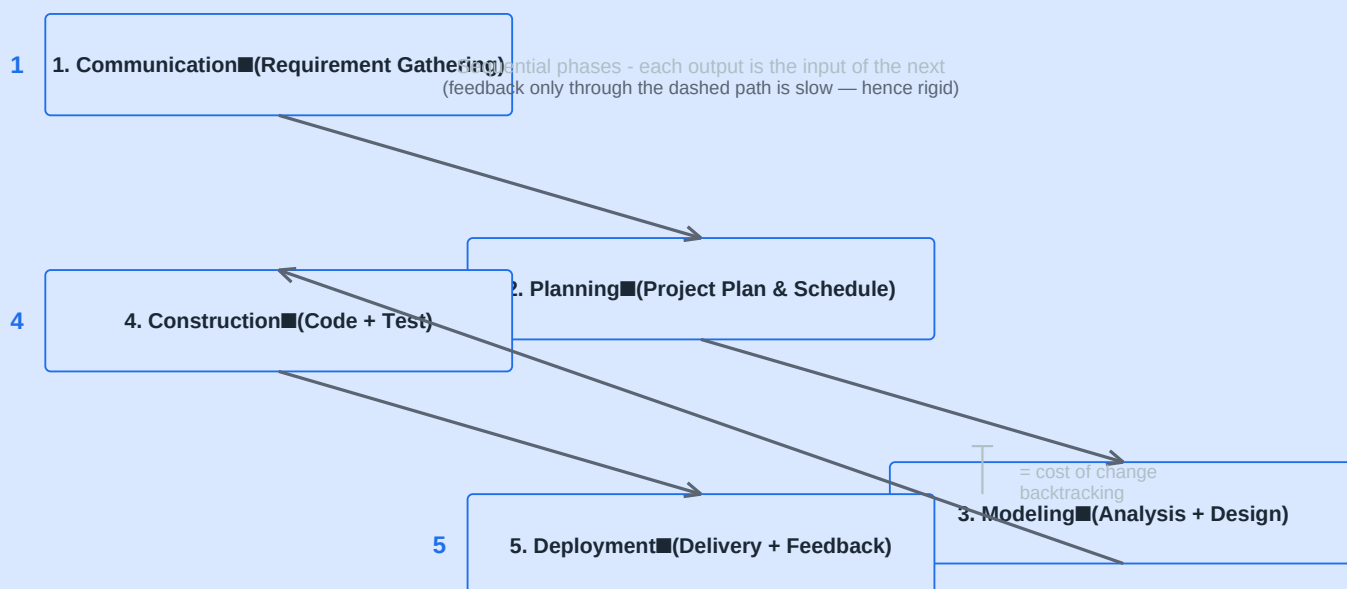
```
// A process is to software what a recipe is to a meal:
Process = Activities + Actions + Tasks + Work products + Quality gates
e.g. Activity : construction
    Action : coding
    Task : 10 tasks of writing methods + compiling
```

Process models (SDLC models) - overview

Model	Strengths	Weaknesses	Best when
Waterfall	simple, staged, documented	late feedback, rigid	stable, small, well-understood
Prototyping	user sees/feels early	prototype mistaken for product	unclear requirements
RAD	very fast, strong reuse	needs modules + team per module	componentisable, time-boxed
Incremental	usable early, lower risk	needs good architecture	large systems, staged delivery

Model	Strengths	Weaknesses	Best when
Spiral	explicit risk handling	complex, costly paperwork	high-risk, large projects
Component assembly	reuses tested parts	component fit mismatch	COTS-heavy systems
RUP	iterative, UML-friendly	heavy process, needs training	large OO enterprise projects
Agile	adapts to change, fast feedback	light doc, needs discipline	changing requirements

Waterfall (Linear-Sequential / Classic) Life-Cycle Model



Waterfall: fully sequential; best for frozen requirements

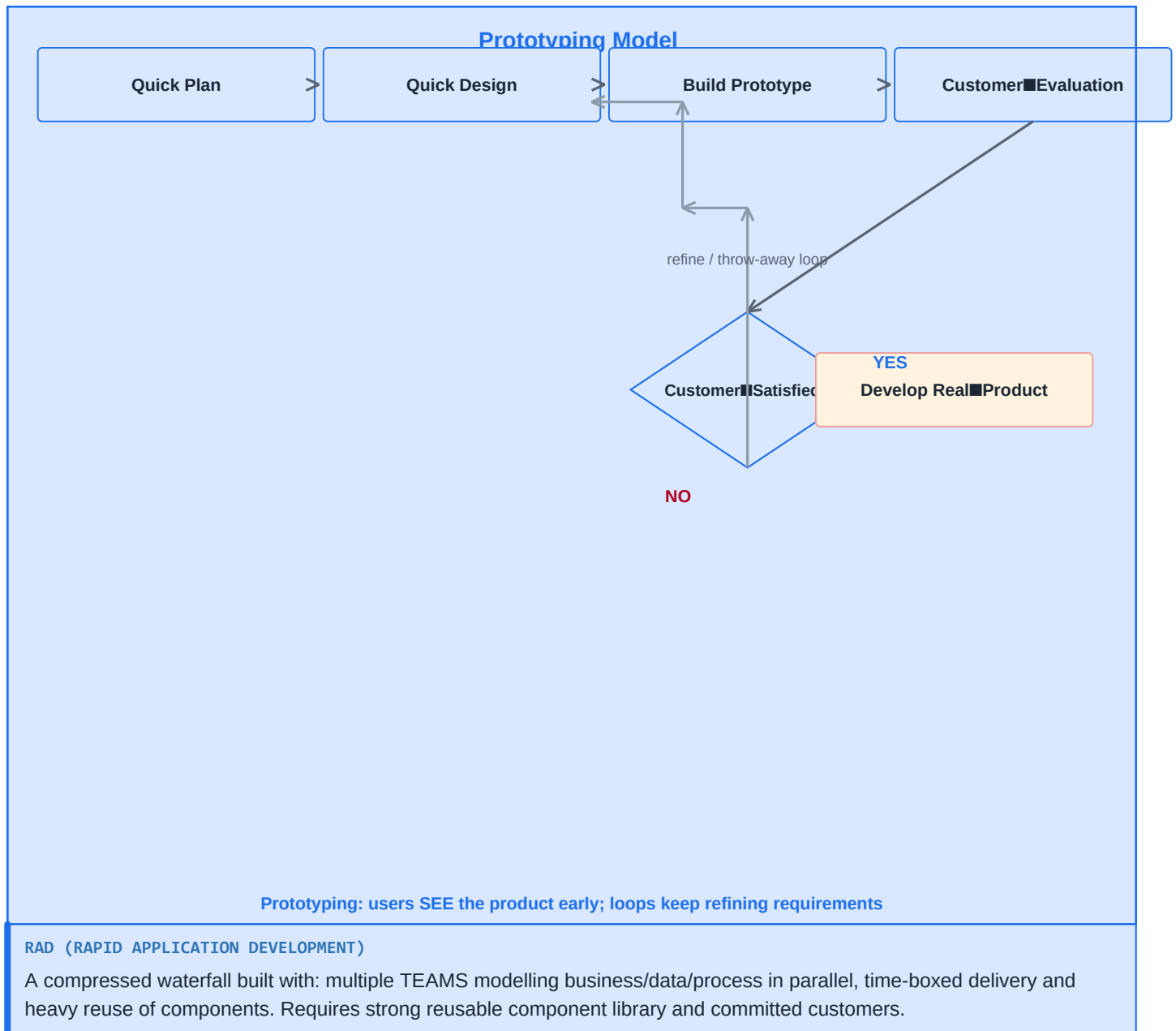
WHEN WATERFALL WORKS / FAILS

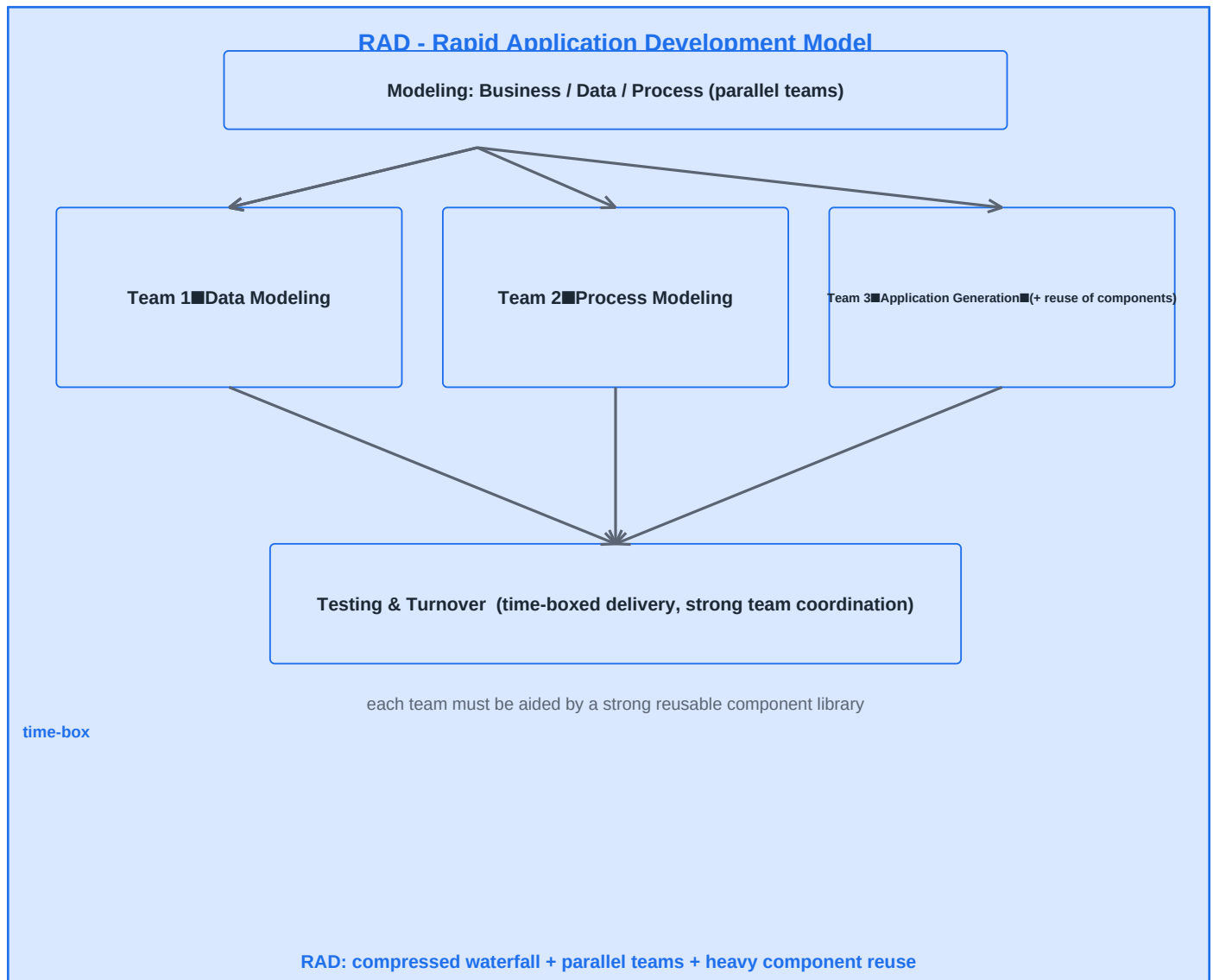
True for: requirements frozen, technology stable, small-to-medium, no user involvement early. Fails for: any uncertainty - problems surface only after coding (cost of change explodes).

1.3 Prototyping & RAD (Linear-Sequential & fast variants)

PROTOTYPING

Build a throw-away or evolutionary working model quickly (UI screens, key logic) so customers can SEE it. Feedback refines requirements. Two flavours: throwaway (discard prototype, write real product) and evolutionary (prototype slowly becomes the product).



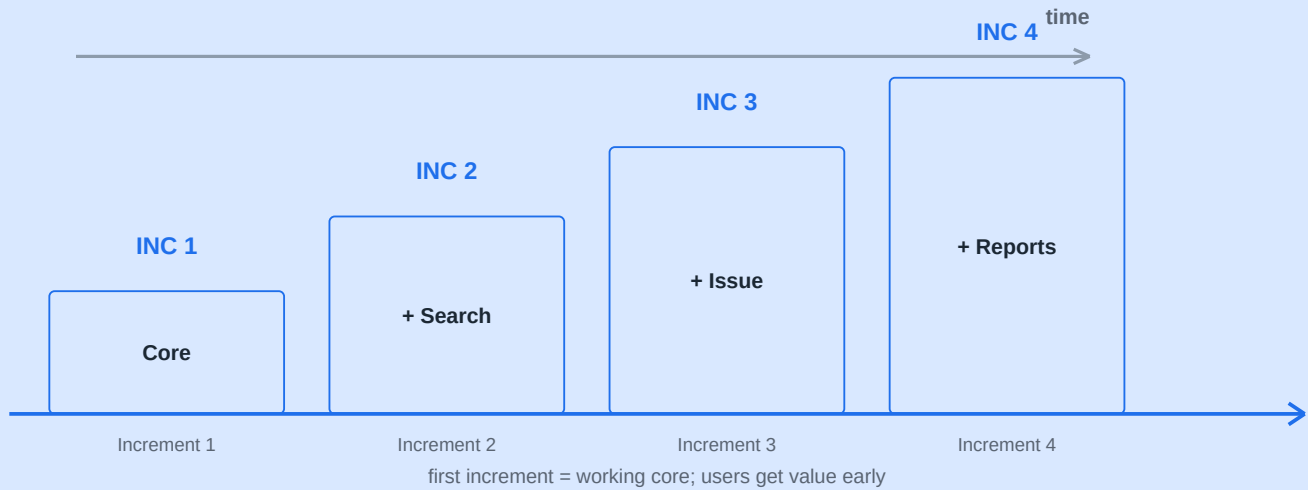


1.4 Evolutionary Process Models

INCREMENTAL MODEL

The product is delivered as a sequence of INCREMENTS. The first increment is a core working product; each following increment adds visible function. Users get value early; risk spreads across increments. (Best with good overall architecture.)

Incremental Model - growing core + features



Incremental: released in in-crement stages - each adds visible function

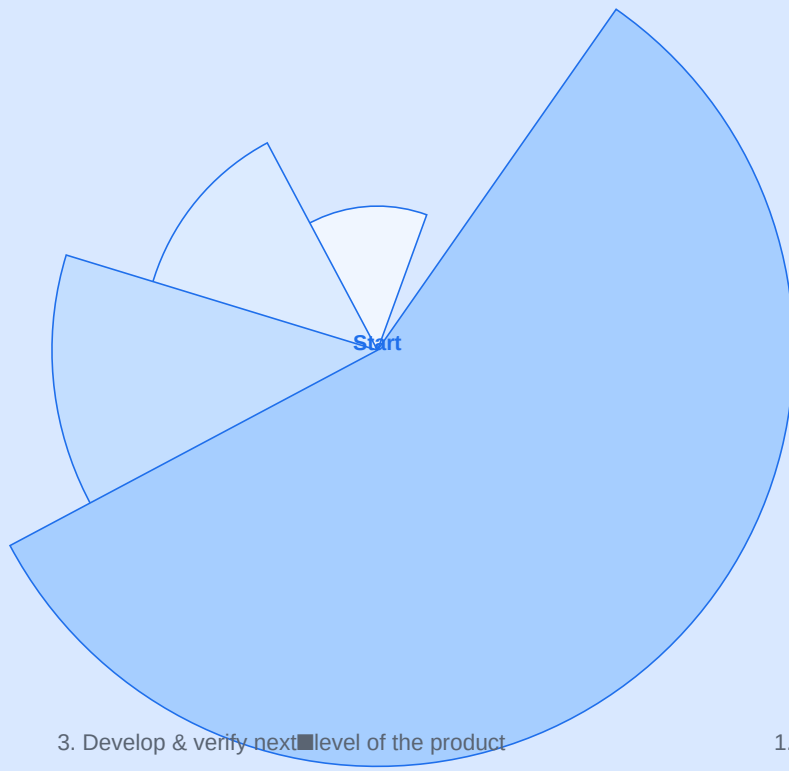
SPIRAL MODEL (BOEHM)

Risk-driven. Each loop goes through: 1) determine objectives & alternatives, 2) identify & resolve risks (prototype/simulation), 3) develop & verify the next-level product, 4) plan the next iteration. Radius grows = more cost per loop.

Spiral Model (Boehm) - risk driven

2. Evaluate alternatives, resolve risks (prototype)

4. Plan the next iteration

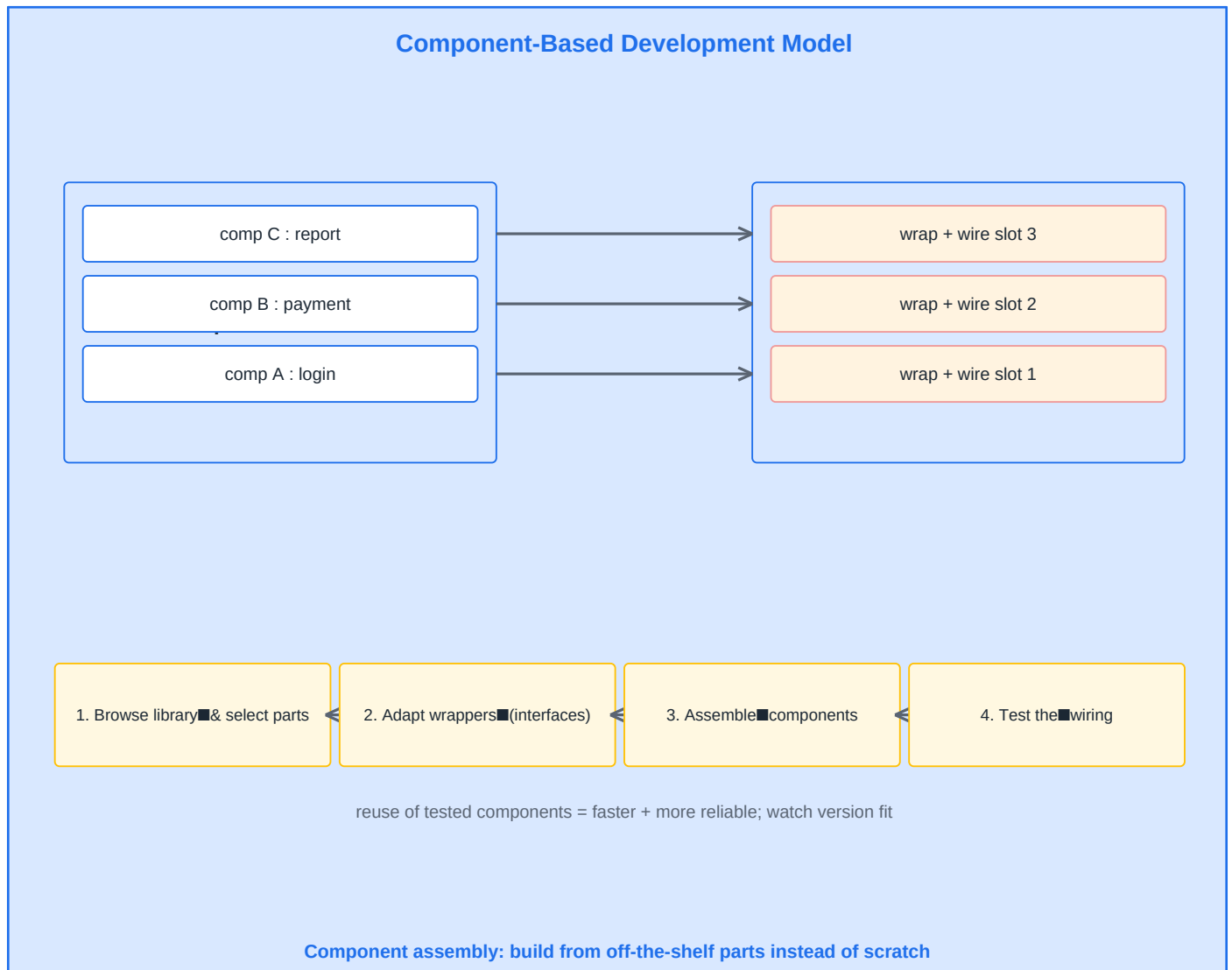


each loop = bigger product + more cost; every loop starts by tackling risk

Spiral: objectives -> risk analysis -> develop -> plan; repeat

COMPONENT-ASSEMBLY MODEL

Build from existing OFF-THE-SHELF components: browse a component library, adapt integration wrappers, assemble, then test the wiring. Delivers fast but risks "component fit" mismatches and maintenance/version dependencies.

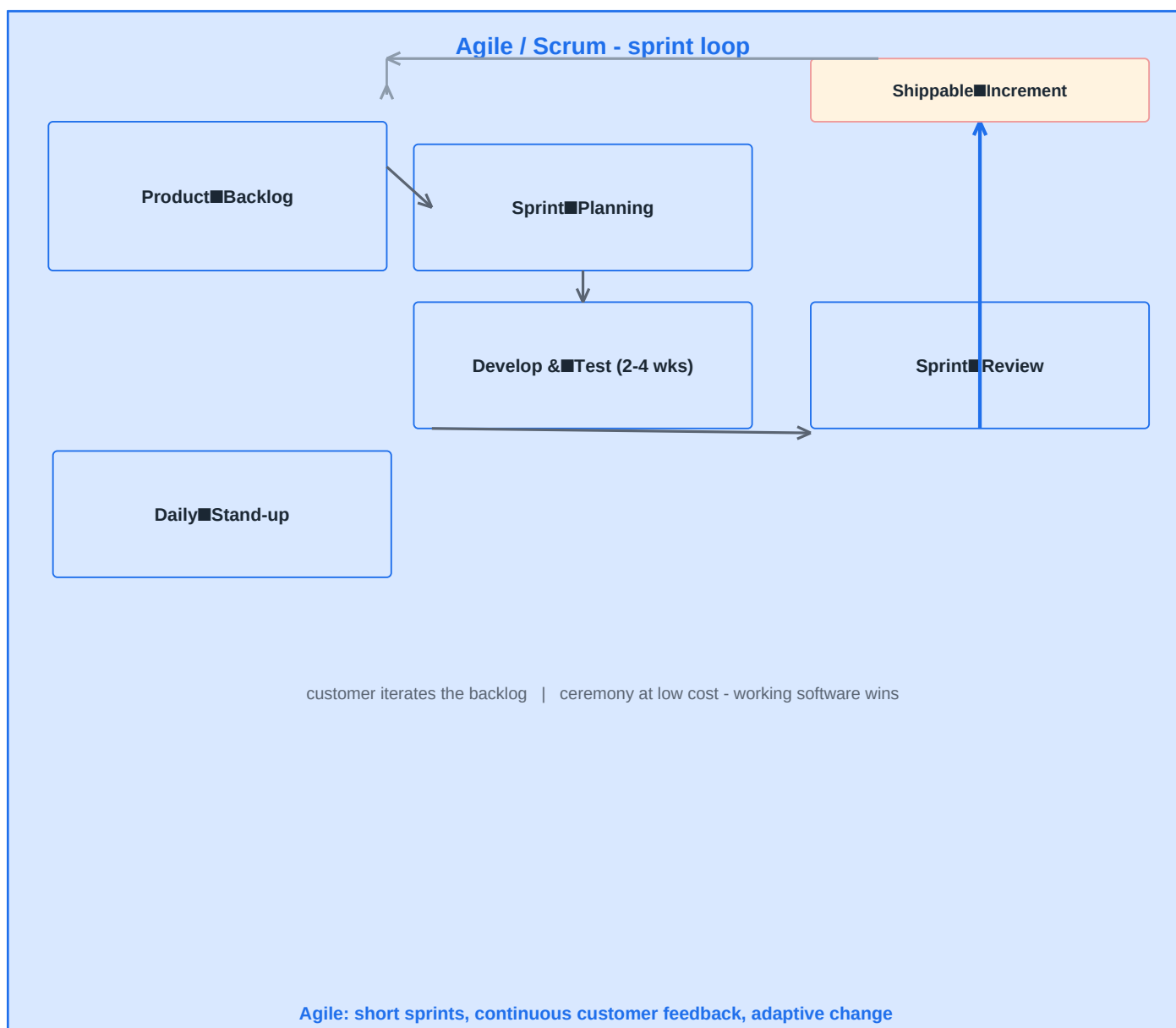


1.5 RUP & Agile processes

RUP (RATIONAL UNIFIED PROCESS)

An iterative, UML-centric process dividing the life-cycle into 4 PHASES - Inception, Elaboration, Construction, Transition - each with iterations. Disciplines flow across phases (requirements, analysis&design, implementation, test, deployment). It is "use-case driven, architecture-centric, iterative and incremental".





Agile principle (Manifesto)	What it replaces	In practice
Individuals & interactions > processes/tools	process-centric teams	stand-ups, pairing
Working software > comprehensive docs	document-heavy phases	demo each sprint
Customer collaboration > contract negotiation	sign-off-culture	on-site customer, backlog
Responding to change > following a plan	frozen plan	re-prioritise each sprint

1.6 Process Customization, Improvement & Metrics

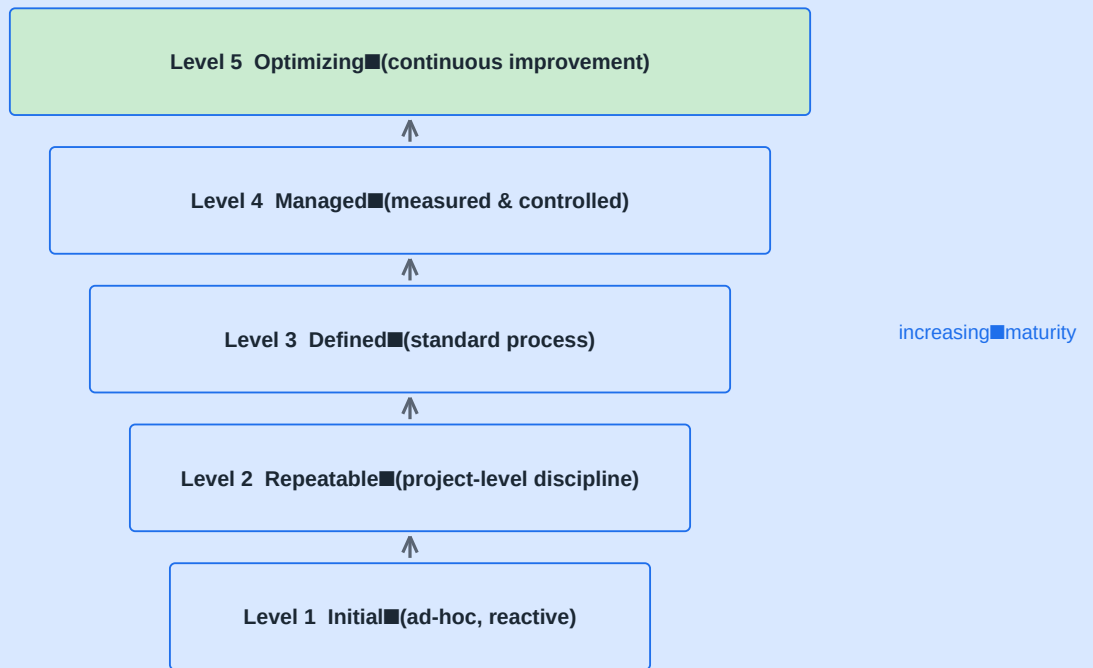
PROCESS CUSTOMIZATION

One process does NOT fit all. Customise by: project size & type (e.g., web app vs embedded), team experience, risk profile, and business environment. A "heavy" plan-driven process suits safety-critical; a "light" agile process suits small dynamic teams.

PROCESS IMPROVEMENT & CMM

Improve the WAY you build software before improving the product. CMM (Capability Maturity Model) ranks an organisation's process maturity in 5 levels: Initial -> Repeatable -> Defined -> Managed -> Optimizing. Higher maturity = predictable, continuously improving process.

CMM - Capability Maturity Model (5 levels)



each level needs process training + enforcement - quality is engineered, not accidental

CMM levels: from chaotic Initial to statistical optimizing process

PRODUCT VS PROCESS METRICS

PROCESS metrics measure the development activity (defects found, time per phase, effort per module) - used to improve. PRODUCT metrics measure the software itself: size (LOC, function points), complexity (McCabe cyclomatic), quality (defect density, availability), design (cohesion/coupling, fan-in/out).

Class	Example metrics	Used to
Process	defect removal efficiency, reuse %, cycle time	improve the process
Product - size	LOC, KLOC, Function points, feature points	estimate effort
Product - quality	defect density, mean-time-to-failure, user satisfaction	judge quality
Product - design	cohesion, coupling, fan-in/out, DIT, NOC	fix design smells
Project	schedule deviation, effort variance, staff-hours	track the project

Exercise: 1) Which model would you choose for a) a railway reservation system, b) a new startup app with unknown features, c) a NASA spacecraft controller? Justify. 2) Draw line-by-line the first 2 increments of a "Library book issue" system. 3) Map Scrum roles onto your class team. 4) List 5 metrics a university could collect to improve its own software lab process.

UNIT 2 | REQUIREMENTS: ELICITATION, ANALYSIS & SPECIFICATION

2.1 Functional & Non-functional Requirements

REQUIREMENT

A condition or capability needed by a user to solve a problem or achieve an objective. It is the **CONTRACT** between customer and developer - the single most common cause of project failure is bad requirements (wrong, incomplete, ambiguous).

FUNCTIONAL REQUIREMENT (FR)

WHAT the system must DO: services, computations, business rules, data processing. Often phrased as "system shall ...". Example: "ATM shall dispense cash only after the PIN is verified."

NON-FUNCTIONAL REQUIREMENT (NFR)

HOW WELL the system does it: performance, security, usability, reliability, availability, constraints (platform, budget, standards). Example: "withdrawal response time must be under 3 seconds; 99.9% availability."

Category of NFR	Concern	Example requirement
Performance	speed, throughput, latency	1000 concurrent users, < 300 ms
Reliability	failure-free behaviour	MTBF > 2000 hours
Availability	uptime percentage	99.9% (≈ 8.7 hrs down/year)
Security	authenticate/authorise, privacy	PCI-DSS, password hashing
Usability	ease of learning/use	new user trains in < 30 min
Maintainability	ease of change	modules traceable to requirements
Portability	move across platforms	runs on Windows & Linux
Legal / ethical	compliance	GDPR, copyright, accessibility (WCAG)

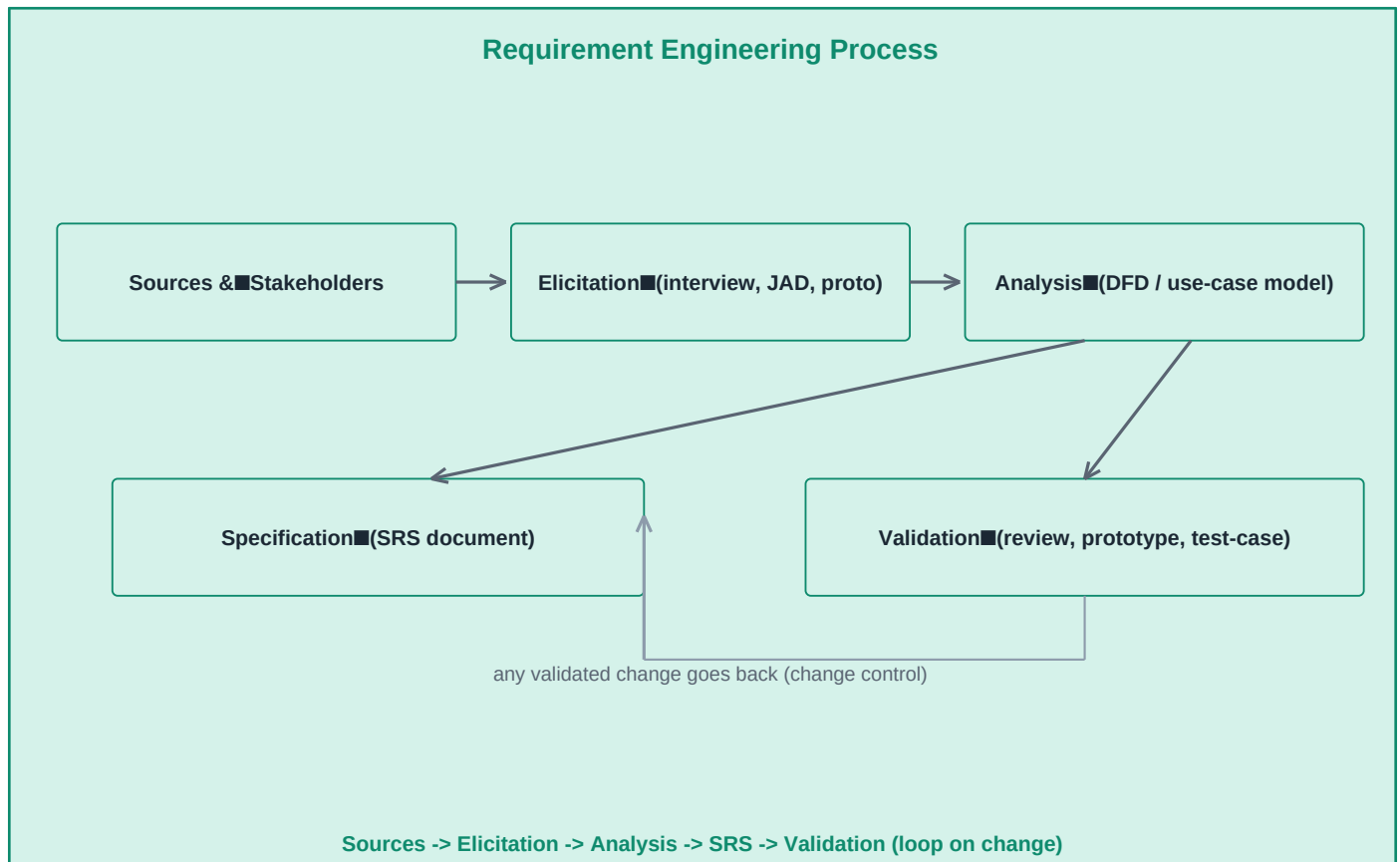
KEY POINT

FURPS+ is the classic NFR memo: Functionality, Usability, Reliability, Performance, Supportability + constraints. Testers convert every NFR into a measurable acceptance criterion (performance test, load test, security scan).

2.2 Requirement Sources & Elicitation Techniques

SOURCES OF REQUIREMENTS

Stakeholders (users, customers, domain experts, operators), existing systems (the SILENT users), documents (forms, reports, procedures), regulations/standards (RBI, ISO, IT Act), and even the competitor's product. Requirement sources must be explicitly identified before elicitation.



Technique	How it works	Strengths	Weakness
Interviews	structured / unstructured Q&A	deep individual views	few people, bias
Questionnaires	written survey of many users	cheap, wide reach	no follow-up depth
Brainstorming	group idea generation	creative, many ideas	needs skilled host
JAD (Joint Application Development)	structured workshop of stakeholders	fast consensus, less rework	expensive to gather experts
Observation / ethnography	watch users at work	real workflows captured	time-consuming
Prototyping	show a working model	users "see" their needs	may be taken as product

2.3 Analysis Modelling — Function vs Object Oriented

ANALYSIS MODEL

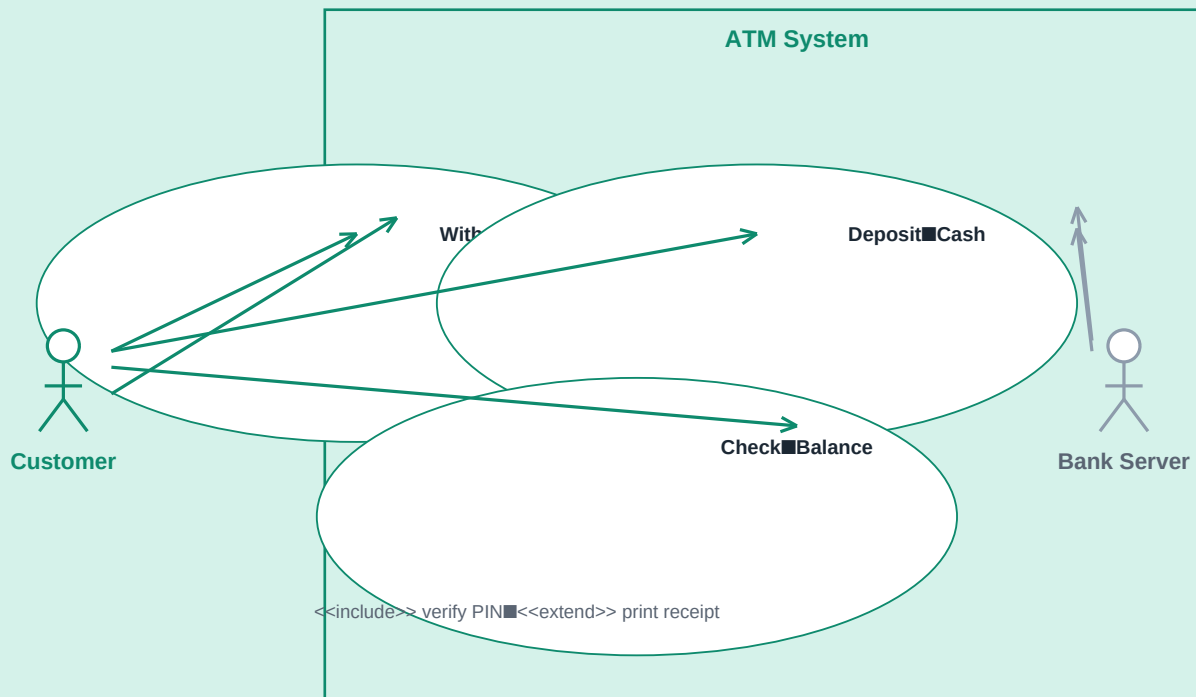
A disciplined picture of WHAT the system must do, independent of HOW. Function-oriented analysis uses Data Flow Diagrams, ER-Diagrams and a data dictionary. Object-oriented analysis uses use-case, class and sequence diagrams (UML). Both must be traceable to the SRS.

Aspect	Function-oriented (SA)	Object-oriented (OO)
Focus	data + functions that change it	objects with state & behaviour
Primary model	DFD, ERD, data dictionary	use-case, class, sequence diagrams
Decomposition	functions -> sub-functions	classes -> responsibilities & relations
Change impact	function change ripples through data flow	localised to an object
Reuse unit	procedure/module	class (data + methods together)
Tools	Yourdon, Gane-Sarson	UML (Rational/StarUML)

2.4 Use-Case Modelling

USE CASE

A scenario of actor interaction that produces a valuable result. USED by analysts to capture functional needs. Components: ACTOR (external role: person or system), USE CASE (ellipse: the goal), SYSTEM BOUNDARY, and ASSOCIATIONS (lines). Preconditions/postconditions and normal & alternate flows describe each use case in detail.

Use-Case Diagram - ATM (abridged)

Use case: actor + ellipse(s) + system boundary = the WHAT

usecase.txt

```
// Use-case description template (for "Withdraw Cash"):
Use Case : Withdraw Cash
Actor : Customer
Precondition : card inserted, PIN verified
Normal flow : 1. Customer enters amount
               2. System checks balance & daily limit
               3. System dispenses cash and updates balance
               4. System prints receipt
Alternate flows : A1 insufficient funds -> show error, return
                  A2 machine out of cash -> notify, cancel
Postcondition : balance reduced, transaction logged
```

2.5 System & Software Requirement Specification (SRS)**SRS**

The approved written statement of requirements - the basis for design, testing and acceptance. IEEE 830 structure: Introduction, Overall description, Specific (functional & NFR) requirements, Appendices. It is the contract; changes after approval go through change control.

SRS_template.txt

IEEE 830 SRS outline:

```

1. Introduction      1.1 Purpose  1.2 Scope  1.3 Definitions  1.4 References  1.5 Overview
2. Overall Description  2.1 Product perspective  2.2 Functions  2.3 User classes
                   2.4 Operating environment  2.5 Constraints  2.6 Assumptions
3. Specific Requirements  3.1 Functional requirements (FR-1, FR-2, ...)
                   3.2 Non-functional (performance, security, usability, ...)
4. (Appendices - data dictionary, algorithms)
Quality of a good SRS: Correct, Unambiguous, Complete, Consistent, Ranked (priority),
Verifiable, Modifiable, Traceable (each item has an ID)

```

VALIDATION VS VERIFICATION

VALIDATION: "are we building the RIGHT product?" - walkthroughs, prototypes, checking with users, acceptance criteria.
 VERIFICATION: "are we building it RIGHT?" - checking SRS against stated rules, consistency, completeness review. Both use the same inspection techniques.

2.6 Requirement Validation & Traceability

REQUIREMENT VALIDATION

A set of checks performed to ensure the SRS is correct & usable: (1) validity of each req (truly needed), (2) consistency, (3) completeness, (4) realism (buildable with budget/tech), (5) verifiability (testable). Techniques: reviews, prototyping, checklist-based inspection, walkthroughs, test-case generation early.

TRACEABILITY

The ability to follow a requirement FORWARD (to design & code & tests) and BACKWARD (from code to its requirement). A requirement traceability matrix (RTM) maps every requirement to design module, code unit, test case and change. Traceability proves coverage and makes impact analysis and change control fast.

Requirement ID	Design module	Code unit	Test case	Origin
FR-1 (login)	AuthService	Auth.java	TC-7 wrong PIN	JAD* session 2
FR-2 (withdraw)	AccountService	Account.java	TC-9 limit test	customer interview
FR-3 (balance)	StatementService	Balance.java	TC-11 print	existing system
NFR-4 (3s response)	all services	profiles	TC-14 load test	SLA with bank

REQUIREMENT TRACEABILITY MATRIX (RTM) IDEA

```

Requirement -----> Design -----> Code -----> Test
FR-1                AuthService        Auth.java        TC-7
|                   |                 |                 |
+----- RTM rows -----+-----+-----+
any change in FR-1: impact = rows of the matrix -> who to re-build & re-test

```

Exercise: 1) List 4 FR and 4 NFR for an online bus-booking site. 2) Plan a JAD workshop agenda (60 min) for that site. 3) Write the use case "Cancel Booking" with normal + 2 alternate flows. 4) Draw an RTM with 3 requirements and trace each to a test.

UNIT 3 | SOFTWARE DESIGN, UML, ARCHITECTURE & UI DESIGN

3.1 The Design Process & Design Principles

SOFTWARE DESIGN

The bridge between requirements (WHAT) and code (HOW). It produces the DESIGN MODEL: architecture, components, interfaces, and data structures. Design must create a solution that is modular, understandable and changeable - and it must be TRACEABLE to requirements.

Principle	Meaning	Payoff
Abstraction	think at the right level, hide detail	tackle complexity
Refinement	stepwise elaboration down to code	correctness by stages
Modularity	split into independent modules	divide & conquer
Information hiding	hide secrets (design decisions)	change without ripple
Functional independence	high cohesion + low coupling	localised change
Good interfaces	clear params, small surface	easy integrate/test

COHESION & COUPLING (THE TWO MOST IMPORTANT QUALITY MEASURES)

HIGH COHESION = a module does ONE well-defined job (functional cohesion is the best). LOW COUPLING = modules talk through clean, narrow interfaces (data coupling best). Rule of thumb: design for "high cohesion, low coupling" - it makes every other quality goal easier.

Cohesion level (weak -> strong)	Coupling level (loose -> tight)
Coincidental - random grouping	Content - one uses inside of another
Logical - "all I/O here"	Common - shared globals
Temporal - things done at same time	Control - flag passes control
Procedural - sequence of a function	Stamp - shares structure
Communicational - share data, one task	Data - only simple params (best)
Sequential - output feeds next	
Functional - single purpose (best)	

3.2 UML - The Unified Modelling Language

UML

A standard set of diagrams for visualising, specifying, constructing and documenting a software system (an OO standard managed by the OMG). STATIC diagrams capture structure; DYNAMIC diagrams capture behaviour/time.

Kind	Diagram	Answers
Static (structure)	Class diagram	what classes & how they relate
Static (structure)	Component diagram	physical software building blocks
Static (structure)	Deployment diagram	hardware + software placement
Behaviour	Use-case diagram	what actors can do with system
Behaviour	Sequence diagram	message order over time
Behaviour	Activity diagram	work-flow / algorithm flow
Behaviour	State diagram	object states & transitions

class_diagram.java.txt

```
// A simple UML Class in 'Java-sketch' form (for UML translate to boxes):
// +-----+
// | Account |
// +-----+
// | - accountNo : String |
// | - balance : double |
// | - owner : Customer |
// +-----+
// | + deposit(amount : double) : void |
// | + withdraw(amount : double) : boolean |
// | + getBalance() : double |
// +-----+
// Relations: Account 1 -- * Transaction ; Account 0..1 --> Customer
```

MINI USE-CASE & RELATION LEGEND (REAL ARROWS DRAWN IN REPORT LAB)

```
Actor -----<> [Use case] ... (extends/ includes) ----->
+----- Customer -----+ +-----+
|                           | | Withdraw Cash <<include>> -> Verify PIN
|                           | +-----+ | Deposit Cash
| Bank Clerk                +-----+ | Open Account
+-----+ +-----+
relationship labels: 1---* composition / inheritance arrows in UML
```

3.3 Architectural Design - views, styles & MVC

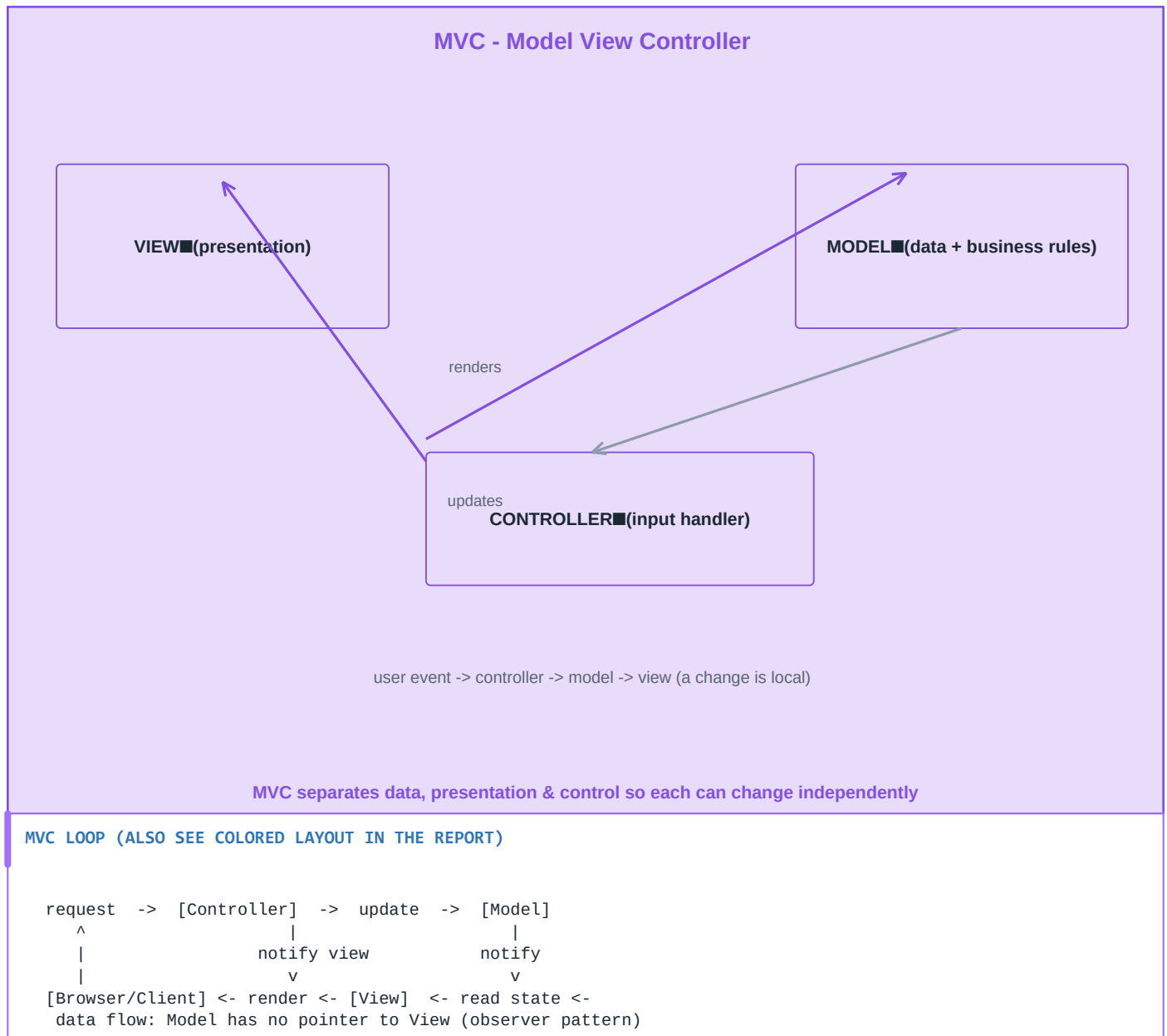
ARCHITECTURE & 4+1 VIEWS (KRUCHTEN)

Architecture = structure + interactions of major components. Kruchten's 4+1: LOGICAL view (OO classes), PROCESS view (concurrency/threads), DEVELOPMENT view (layers & modules), PHYSICAL view (hardware deployment) + SCENARIOS (use cases) that glue them together.

Architectural style	Structure	Used for
Data-centred (repo)	central data store + client processes	information systems, DB apps
Data-flow (pipe-filter)	streams transformed by filters	compilers, ETL, Unix tools
Call-and-return / layered	vertical layers call below	OS, enterprise apps
Object-oriented	objects encapsulate data + services	generic problem domains
Layered / MVC	Model - View - Controller split	GUI apps, frameworks
Client-Server	thin client <-> server	web and network apps

MVC

MODEL = data + business rules (no UI), VIEW = what the user sees (renders the model), CONTROLLER = input handling that updates model & re-renders view. Separation lets one model have many views (mobile + web + desktop) and changes stay localised.



3.4 User Interface (UI) Design

GOOD UI = USABLE UI

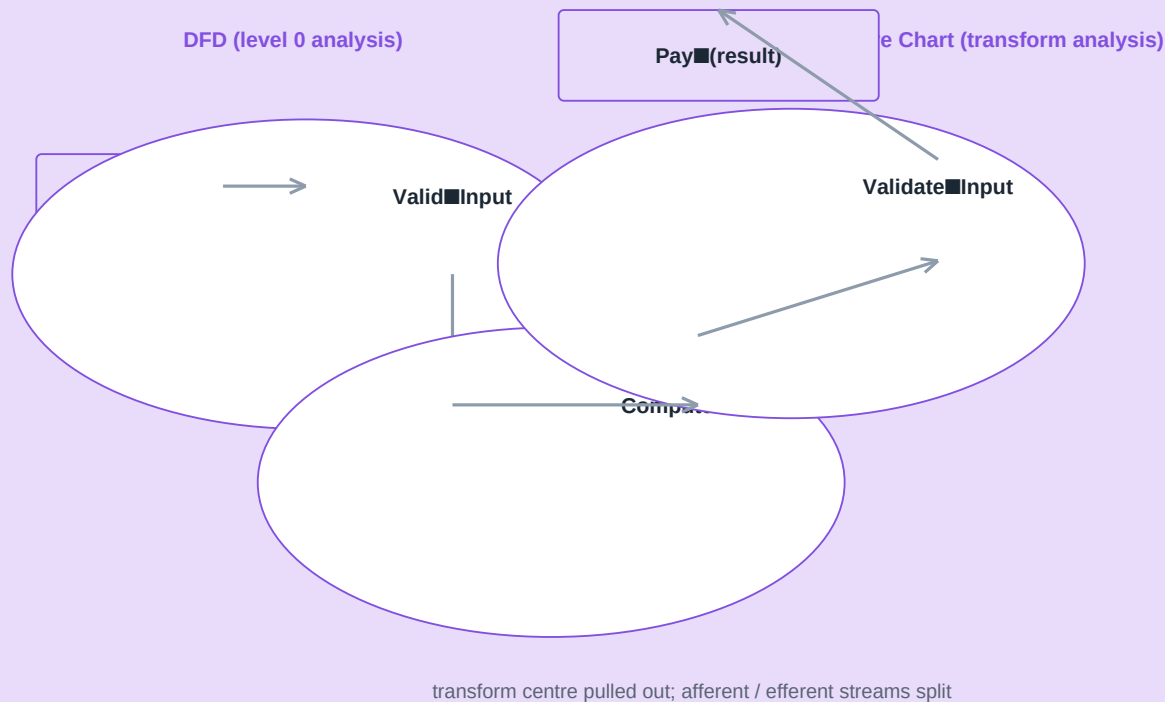
UI design is a blend of art + engineering: users must achieve goals with min effort. Golden rules: place the user in control (Let them UNDO), reduce the user's memory load (recognition over recall), make the interface consistent (same words = same meaning).

Rule	Example
User in control	Cancel / Undo / customisable menus
Reduce memory load	dropdowns, autocomplete, defaults
Consistency	same icon meaning across screens
Feedback	toast after save, progress bar
Error handling	clear msg + how to fix, no error codes
Aesthetics	Fitts law: big targets, white space

3.5 Function-Oriented Design - SA/SD + DFD

SA/SD (YOURDON & CONSTANTINE)

STRUCTURED ANALYSIS builds a DFD + data dictionary, STRUCTURED DESIGN converts the DFD into a structure chart of modules. A DFD shows PROCESSES (circles), DATA FLOWS (arrows), DATA STORES (open boxes) and EXTERNAL ENTITIES (rectangles); it has levels 0 (context) -> 1 -> 2 (FURTHER detail).

SA/SD - from DFD (function-oriented) to Structure Chart

DFD flows -> central transform -> structure chart of balanced modules

TRANSFORM VS TRANSACTION ANALYSIS

TRANSFORM analysis: split a central TRANSFORM centre (the core computation) from its AFFERENT (input) and EFFERENT (output) streams into a balanced structure chart. TRANSACTION analysis: when one action selects among many sub-actions (menu/router), organise into a dispatch module calling one of several transaction modules.

DATA DICTIONARY

A central catalogue of every data item shown in the DFD: name, type, length, values, where used. Notations: = composed of, + and, | or, {} repetition, () optional. The dictionary makes the analysis model exact.

data_dictionary.txt

Data dictionary entries (for an ATM system):

```
account      = accountNo + pin + balance
transaction  = { deposit | withdrawal | inquiry }
withdrawal   = amount(100..10000) + timestamp + receipt
accountNo    = 4..6 digits
```

Every name used in the DFD must resolve in the dictionary (consistency check).

3.6 Component-Based Design & Design Metrics

COMPONENT-BASED DESIGN

The system is assembled from INDEPENDENT, replaceable software components (objects/COTS/services) that communicate through well-defined INTERFACES. Characteristics: replaces a part without rebuilding all, strong encapsulation, interface-first design, explicit deployment. Risks: market dependence on the component vendor, component versioning.

Metric	Formula / meaning	Design message
Fan-out	modules a module calls	too high = complex, hard to test
Fan-in	modules that call it	high = much reuse (good)
Depth in tree (DIT)	inheritance distance from root	deep trees are hard to understand
Cyclomatic v(G)	edges - nodes + 2	base test paths, complexity
Cohesion / coupling	intramodule bind / intermodule bind	target high cohesion + low coupling
Halstead length	$N = N1 + N2$ (operators+operands)	implied code volume & effort

Exercise: 1) Draw the class diagram (boxes) for a "Student — Enrolment — Course" system with multiplicity. 2) Identify the cohesive bundle and couplings in a 4-module ATM. 3) Give 2 style choices for a food-delivery app and match them to golden rules. 4) From the ATM DFD, name 3 processes, 2 stores and 2 external entities.

UNIT 4 | ANALYSIS & SOFTWARE TESTING

4.1 Static Analysis, Inspections & Reviews

STATIC VS DYNAMIC ANALYSIS

STATIC analysis examines the code/model WITHOUT executing it (reviews, linting, data-flow analysis) - it detects defects early using low cost. DYNAMIC analysis RUNS the program against test cases (unit/integration/system tests) and observes behaviour.

Aspect	Static analysis	Dynamic analysis (testing)
Executes code?	No	Yes
Phase	as early as review time	after code exists
Finds	style, syntax, missing cases, smells	runtime failures, wrong outputs
Cost	cheap, fastest defect removal	needs test env + cases
Examples	code review, walkthrough, lint, SonarQube	unit test, Selenium, load test

INSPECTIONS & REVIEWS

FAGAN inspection: a formal team meeting where code/SRS is checked line-by-line against a checklist - defects are logged (NOT fixed at the meeting) and re-inspected after fix. WALKTHROUGH: author leads an informal walk; REVIEW: managers/peers evaluate against goals. Review removes 60-90% of defects before testing starts.

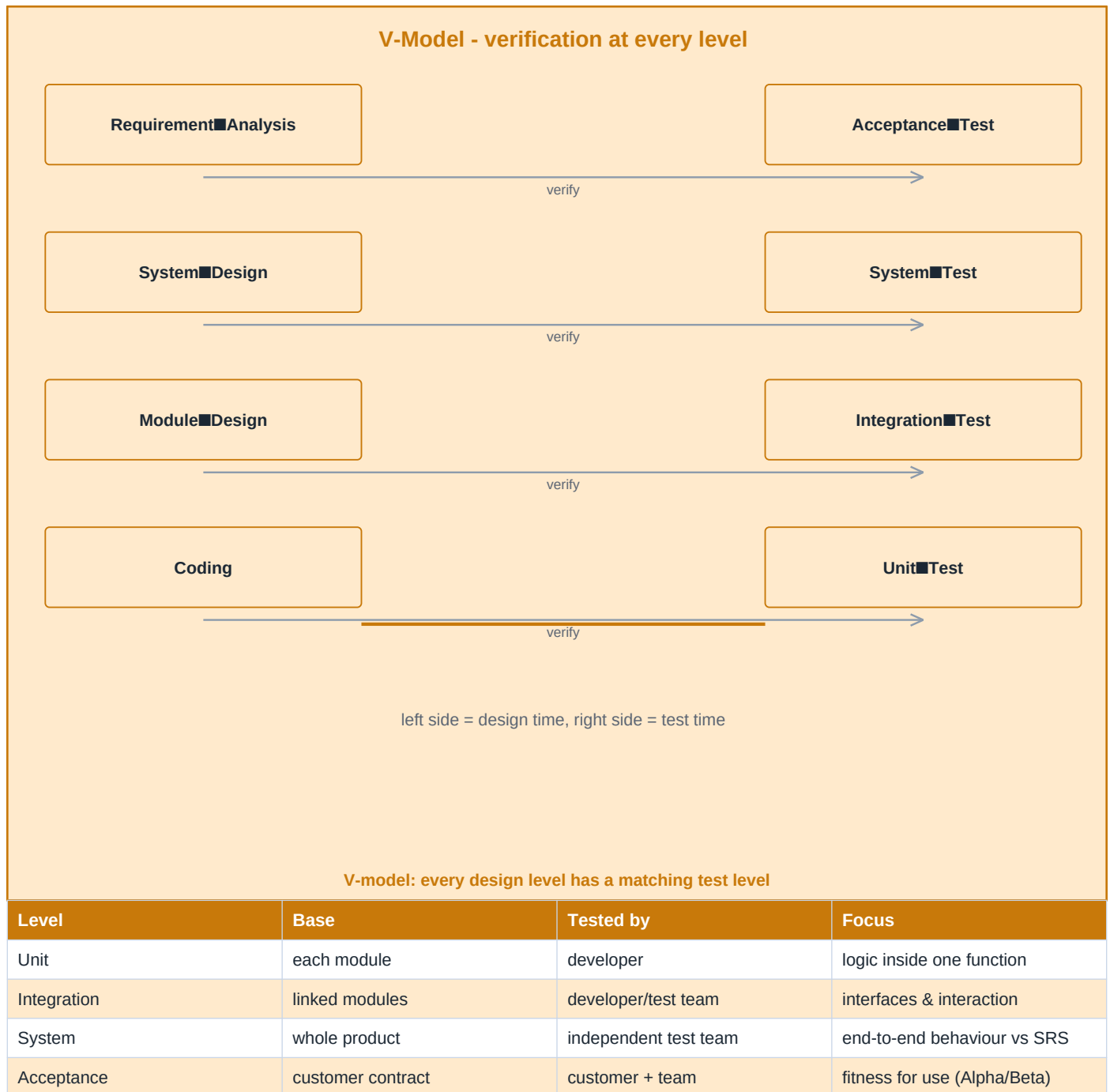
FAGAN INSPECTION PROCESS

Plan meeting -> Overview -> Preparation (checklist) -> Inspection meeting ->
 Rework (author fixes) -> Follow-up / re-inspect (exit when clean)
 Roles: Moderator, Author, Readers, Recorder (defect list), Tester
 data: defect type, severity, phase injected, phase detected (for RCA & cost of defect)

4.2 Testing Fundamentals & V-Model

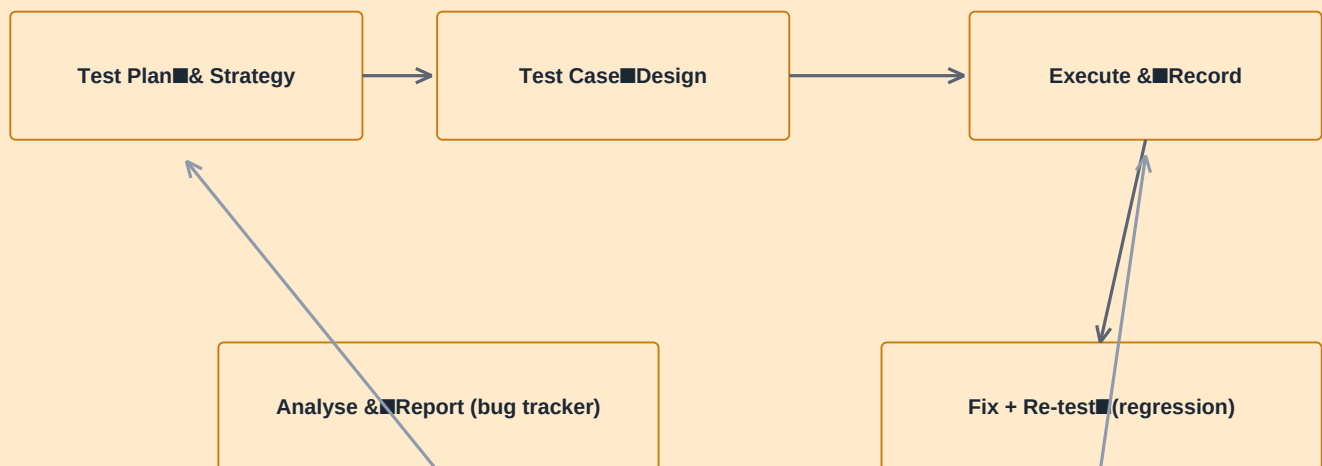
TESTING = VERIFICATION OF SOFTWARE

Executing a program with the intent of FINDING ERRORS (Good) - a test is successful only if it DISCOVERS a defect. Testing verifies the product against requirements; DEBUGGING is the separate activity of locating and correcting the confirmed fault. Note: exhaustive testing is impossible - we sample input classes smartly (black-box & white-box).



4.3 Test Process & Test Case Design (Black-Box)

Software Testing Life-Cycle (STLC)



every defect traced to test case - metrics: pass %, DRE, density

Plan -> design -> execute -> analyse -> fix & regress (full loop)

BLACK-BOX (BEHAVIOURAL) TESTING

Tests the system's behaviour from OUTSIDE - no look at internals. Inputs derived from requirements spec. Techniques: equivalence partitioning, boundary value analysis, decision tables, state-transition testing, error guessing, random/regression testing.

Technique	Idea	ATM example
ECP (Equivalence Partitioning)	partition valid/invalid classes, test one from each	amount classes: <100, 100-10000, >10000
BVA (Boundary Value Analysis)	test the EDGES of each class (min-1, min, min+1, ...)	99, 100, 101, 9999, 10000, 10001
Decision Table	matrix of conditions -> actions	PIN ok? card ok? cash avail? -> 6 rules
State Transition	model events between states	Idle->Active->Blocked on 3 wrong PINs
Error guessing	experience-based brain-storming	empty amount, decimal comma, re-insert card

BLACK-BOX : ECP + BVA + DECISION TABLE + STATE TRANSITION AT A GLANCE

number-of-items : [-] valid [0] invalid <=0 [1..15] valid cap [>15] invalid > cap
 * boundary tests: -1, 0, 1 | 14, 15, 16 (BVA picks 2k-1 points per boundary)
 * decision table row: (PIN ok=T, balance ok=T, cash ok=T) => action dispense
 * state diagram: [Idle] --insert--> [Active] --x3 wrong--> [Blocked]

4.4 White-Box (Structural) Testing & Cyclomatic Complexity

WHITE-BOX TESTING

Tests the INTERNAL logic: every statement, branch, condition and path. Used for unit testing and to measure COVERAGE. Coverage grows: statement -> branch -> condition -> path (path is impractical in general; path count can be exponential - so we bound it with paths from structured analysis).

Coverage criterion	Requires that...	ATM example
Statement	every executable statement is run	run both return branch bodies
Branch/decision	every decision takes T and F at least once	if(amount<=balance) both ways
Condition	every atomic condition in a compound disjunction	amount<=balance && pinOk all combos
Path / basis set	every linearly independent path via v(G)	v(G) independent paths

Login.java

```
// v(G) cyclomatic complexity = E - N + 2P (E edges, N nodes, P=1 for one module)
// never exceed 10 per function (rework/refactor beyond that).
int login(String user, String pin, int tries) { // N=6 nodes after parsing
    if (user == null) return -1; // 1 branch
    if (tries > 3) return 0; // 2nd branch -> v(G) = E-N+2
    if (pin.equals(db.find(user))) return 1; // 3rd branch -> v(G) = 4
    return -2; // paths: one per basis path (4)
}
```

TEST CASE DESIGN DOC (MANDATORY FOR EVERY TEST)

Each test case carries: TC-ID, requirement link, preconditions, inputs, expected output/state, and trace to technique (ECP/BVA/...). This makes tests repeatable, reviewable and automatically measurable.

Test Case	Pre-condition	Inputs	Expected result
TC-01 (ECP valid)	card valid, PIN ok	amount = 5000	cash + receipt, balance drops
TC-02 (BVA low)	PIN ok	amount = 99	error "on limit reached", no cash
TC-03 (state)	3 wrong PIN in a row	4th attempt any	card blocked, no retry
TC-04 (regression)	after balance fix	full old suite	no behaviour changed

4.5 Unit Testing with JUnit & Integration Strategies

AccountTest.java

```
import static org.junit.Assert.*;
import org.junit.Test;
public class AccountTest {
    @Test public void withdrawOk() {
        Account a = new Account("ACC1", 5000.0);
        assertTrue("should withdraw", a.withdraw(1000.0));
        assertEquals(4000.0, a.getBalance(), 0.001);
    }
    @Test(expected = InsufficientFundsException.class)
    public void withdrawOver_Throws() {
        new Account("ACC1", 500.0).withdraw(1000.0);
    }
}
// run: mvn test -> green bar = suites pass (TDD: write test first, watch it fail, then code)
```

INTEGRATION STRATEGIES

TOP-DOWN: test high-level modules first, stubs stand in for lower ones (exposes control logic early). BOTTOM-UP: test leaf modules with drivers first, climb up (logic issues surface late). SANDWICH: combine both, test middle-out. Each step adds a "build" that catches interface mismatches as modules are joined.

Strategy	Starts at	Uses	Exposes early	Risk
Top-down	high control modules	STUBS for missing tails	interface/control logic	deep logic tested late
Bottom-up	leaf utility modules	DRIVERS for missing heads	leaf logic	big-bang join at end

Strategy	Starts at	Uses	Exposes early	Risk
Sandwich	middle layer	stubs + drivers together	balanced	complex to plan

4.6 System, Specialised & Alpha/Beta Testing

SYSTEM & SPECIALISED TESTING AFTER INTEGRATION

RECOVERY: system must recover from crashes/data loss. SECURITY: penetration attempts resisted (auth, overflow, SQLi). STRESS: pushed beyond limits (many users, full disk) - when does it break? PERFORMANCE: response & throughput under load. USABILITY: real users complete real tasks. COMPATIBILITY: browsers, OS, devices. Each matches an NFR from the SRS.

Type	Question it answers	Typical tool
Recovery	does it come back after a fault?	fault injection scripts
Security	can it be broken into?	OWASP ZAP, metasploit
Stress	where does it collapse?	JMeter, Locust
Performance	fast enough under load?	JProfiler, Apache bench
Usability	can users actually use it?	task observation, SUS score
Compatibility	runs everywhere promised?	browser matrix, VM farm

ALPHA & BETA ACCEPTANCE

ALPHA: on-site test by a controlled user group at the developer's site. BETA: free release to real users in the real environment to find defects you capped not. Both feed defects back before final delivery; acceptance CRITERIA come straight from the SRS validation section.

4.7 Test Plan, Metrics & Tools

TEST PLANNING SUITE (IEEE 829) + METRICS + TOOLS

Test plan (IEEE 829): scope & risks, strategy (levels/techniques), entry/exit criteria, schedule, resources, pass/fail definition, defect reporting process, tools, deliverables.

Metrics: #test cases run, pass %, defect density (defects/KLOC), defect removal efficiency (DRE) = defects removed before release / total

Tools: JUnit (unit), Selenium (web UI), Postman (API), JMeter (load), SonarQube (static), GitLab CI / Jenkins (pipeline automation)

4.8 Structured (SA/SD) vs OOAD - testing view + OO testing

Aspect	Structured / procedural	OOAD / OO
Base unit	procedures & modules	classes & methods with state
Unit test	function in isolation	test class through its public interface
Integration	function call chains	class collaborations & message passing
Reuse test	n/a (copy-paste)	inherited methods need re-test in subclass context
State testing	rare	state-based: object goes thru states
Approach	big-bang or stepwise	thread/use-case-based, + regression by classes

OO TESTING SPECIFICS

CLASS TESTING: test each method of the class + the STATE space of the object (states: empty, active, blocked). INTERCLASS testing: test collaborations among classes (e.g., Account & Transaction). Inherited methods are NOT re-tested in the parent but may need new test cases in a subclass (overridden getters, new invariants).

Exercise: 1) For a "temperature in [18..30]" field, give ECP classes + 6 BVA values. 2) Compute $v(G)$ of a function with 4 if-else nodes (draw flowchart, count E, N). 3) Assign 4 specialised system-test types to an online bank. 4) Plan a 3-class interclass collaboration test for Customer -> Account -> Transaction.

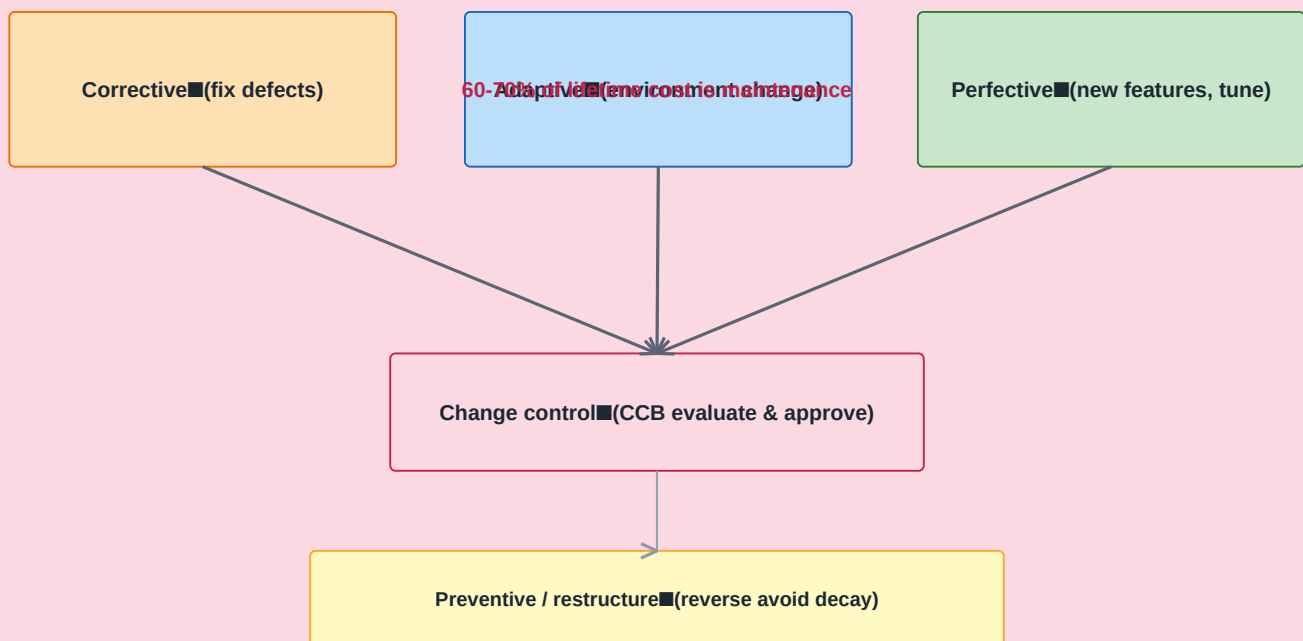
UNIT 5 | MAINTENANCE, SCM & PROJECT MEASUREMENT

5.1 Maintenance - need, types & factors

SOFTWARE MAINTENANCE

Modification of a software product AFTER delivery to correct faults, adapt to environment or improve. Maintenance consumes 60-70% of a software system's total cost - so design-time decisions (maintainability) drive lifetime cost. Legacy systems are large ageing programs still running critical business - the most risky to change.

Software Maintenance - 4 types feeding change



Corrective + Adaptive + Perfective (+ Preventive) all go through change control

Maintenance type	Why it happens	Share of total maintenance (~)
Corrective	fix reported defects/bugs	17-21%
Adaptive	environment change (OS, hardware, laws)	18-25%
Perfective	new/enhanced features, tuning	50-60%
Preventive	remanufacture/restructure to avoid decay	~4-5%

FACTORS THAT DECIDE MAINTENANCE SIZE & SEVERITY

Number of latent defects, design quality (coupling!), documentation quality, staffing turnover, test coverage available, age of structure (revisit old code), and the number of reproductions of the system at sites. Poor "maintainability" turns every perfective change into a regression minefield.

5.2 Software Configuration Management (SCM)

SCM

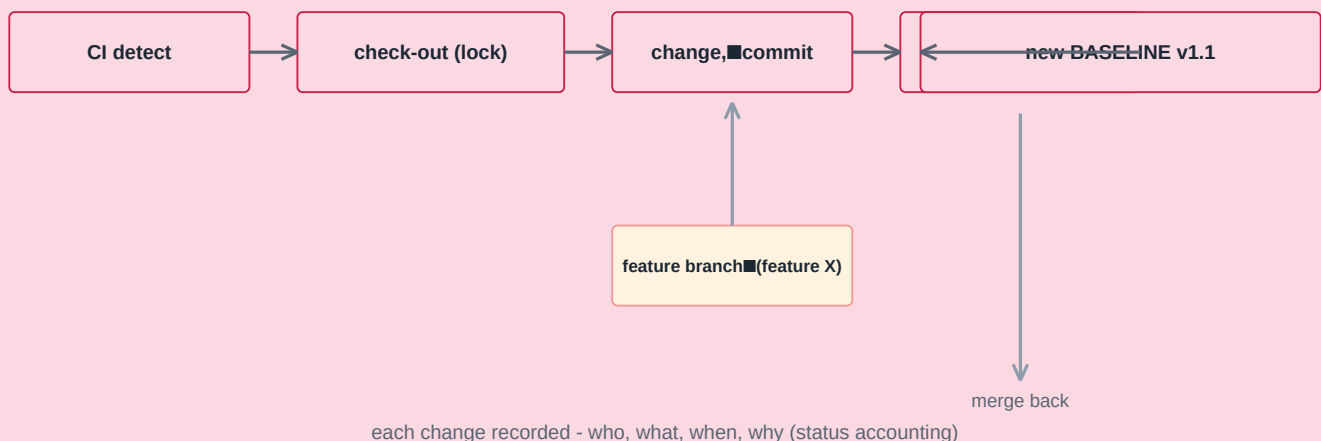
The discipline of controlling EVOLUTION of a software product: identify Configuration Items (CI), control changes, record/report status, and AUDIT configurations. It answers "what version, is it approved, who changed what, when?" SCM is the PROJECT's memory - critical for teams, releases & audits.

```
git_commands.sh
```

```
# git - the pragmatic SCM/version-control tool
git init                # create a repo (identify CIs)
git add src/*.java      # stage changes
git commit -m "fix #42 bal" # record a version (baseline)
git branch feature-login # isolate parallel work
git merge feature-login  # reintegrate after review
git tag v1.0             # freeze a RELEASE baseline
git log --oneline        # status/audit reporting
git blame Login.java     # who changed each line (change records)
```

VERSION CONTROL CONCEPTS

VERSION = a specific state of a CI, labelled (v1.0, build 42). BASELINE = an approved, frozen set of CIs that becomes a reference for change (e.g., the "tested" release). BRANCH = a copy/line of development so parallel work doesn't corrupt the shared baseline; MERGE recombines. One baseline per release; every change = new version + audit trail.

Version Control / Configuration Items flow

Identify CIs -> version each change -> baseline & report (SCM cycle)

CHANGE CONTROL & CCB

CHANGE CONTROL: propose change -> evaluate impact (cost/schedule/risk) -> approve/reject by the CHANGE CONTROL BOARD (CCB) -> implement -> introduce into baseline. Without it, unmanaged change equals unmanaged project. STATUS REPORTING: who did what, when, why; AUDIT: verify that documentation == configuration.

SCM element	Deliverable / record	Tool support
Identification	CI list, naming standards (v1.2.3)	git tags, VSS labels
Change control	ECR/ECP workflow, CCB minutes	JIRA, GitHub PR
Status accounting	release notes, version table	git log, CI dashboards
Auditing	baseline vs doc comparison	diff + review checklists

5.3 Software Re-engineering & Legacy Systems

WHY RE-ENGINEER

Legacy systems are correct & trusted but expensive to maintain (orphan skills, old tech). Re-engineering = cost-effective revival: RESTRUCTURE (improve internal structure, keep behaviour), REVERSE ENGINEER (recover design/specs from code), FORWARD ENGINEER (re-implement a new improved version). Always done incrementally - never as a "big-bang rewrite".

RE-ENGINEERING PIPELINE

```

[Legacy Code] --reverse engineering--> [Abstract Spec] --forward engineering--> [New System]
    |                               |                               |
    (understand)                   (redesign + new SRS)             (fresh impl)
Restructuring keeps functions, improves structure : spaghetti SQL -> normalised + services

```

5.4 Project Management - concepts, planning, feasibility

PROJECT MANAGEMENT (PM)

Planning, organising, staffing, directing and controlling a software project so that scope, cost, time and quality are all met. The manager juggles the PROJECT's 4 constraints: scope, time, cost, quality - change one and the others move.

Management activity	Key questions / outputs
Scoping & planning	what is in/out? -> WBS, project plan, milestones
Estimation	scheduling feasibility, if manager estimates -> effort & cost
Risk management	what can go wrong? -> register + mitigation
Monitoring & control	track progress, metrics, corrective action
Staffing & team	roles, skills, team structure
Review/reporting	status reports, lessons learned

FEASIBILITY STUDY (ANSWERS "SHOULD WE BUILD IT?")

TECHNICAL (is the technology/team available?), ECONOMIC (cost-benefit, ROI, payback period), OPERATIONAL (will it work with people & processes?), SCHEDULE (can we meet the deadline?), LEGAL & ETHICAL (contract, data laws). A NO on any major axis = stop early and cheap.

Feasibility	Checks	ATM example
Technical	skills, tools, platforms	card reader API exists?
Economic	cost vs benefit (NPV, payback)	hardware ROI in 18 months?
Operational	users/org acceptance	bank staff willing & trained?
Schedule	deadline realistic with EFF(-estimate)	can we ship before contract date?
Legal	compliance, licensing	RBI & data-privacy compliant?

5.5 Estimation - LOC, Function Points & COCOMO

SIZE-BASED EFFORT ESTIMATION

EFFORT drives cost & schedule. Two of the most used size measures: LOC (lines of code) and FUNCTION POINTS (user-perceived functionality, language-independent: inputs, outputs, inquiries, files, interfaces each weighted + adjustment factors). FP -> convert to LOC via language table, then apply productivity (~KLOC per person-month).

function_points.txt

```
Function Point count (illustrative ATM):
External inputs (getPIN, deposit, withdraw, balance) ..... 4 x 4 = 16
External outputs (receipt, statement, error-screen) ..... 3 x 5 = 15
External inquiries (mini-statement) ..... 1 x 4 = 4
Internal logical files (account, transaction) ..... 2 x 10 = 20
External interface files (bank-core gateway) ..... 1 x 7 = 7
Unadjusted FP = 62 ; multiply by value-adjustment (0.65..1.35) -> FP
Loc estimate = FP x language factor (e.g. Java ~53 LOC/FP) = 62 x 53 ~ 3286 LOC
```

COCOMO (CONSTRUCTIVE COST MODEL, BASIC)

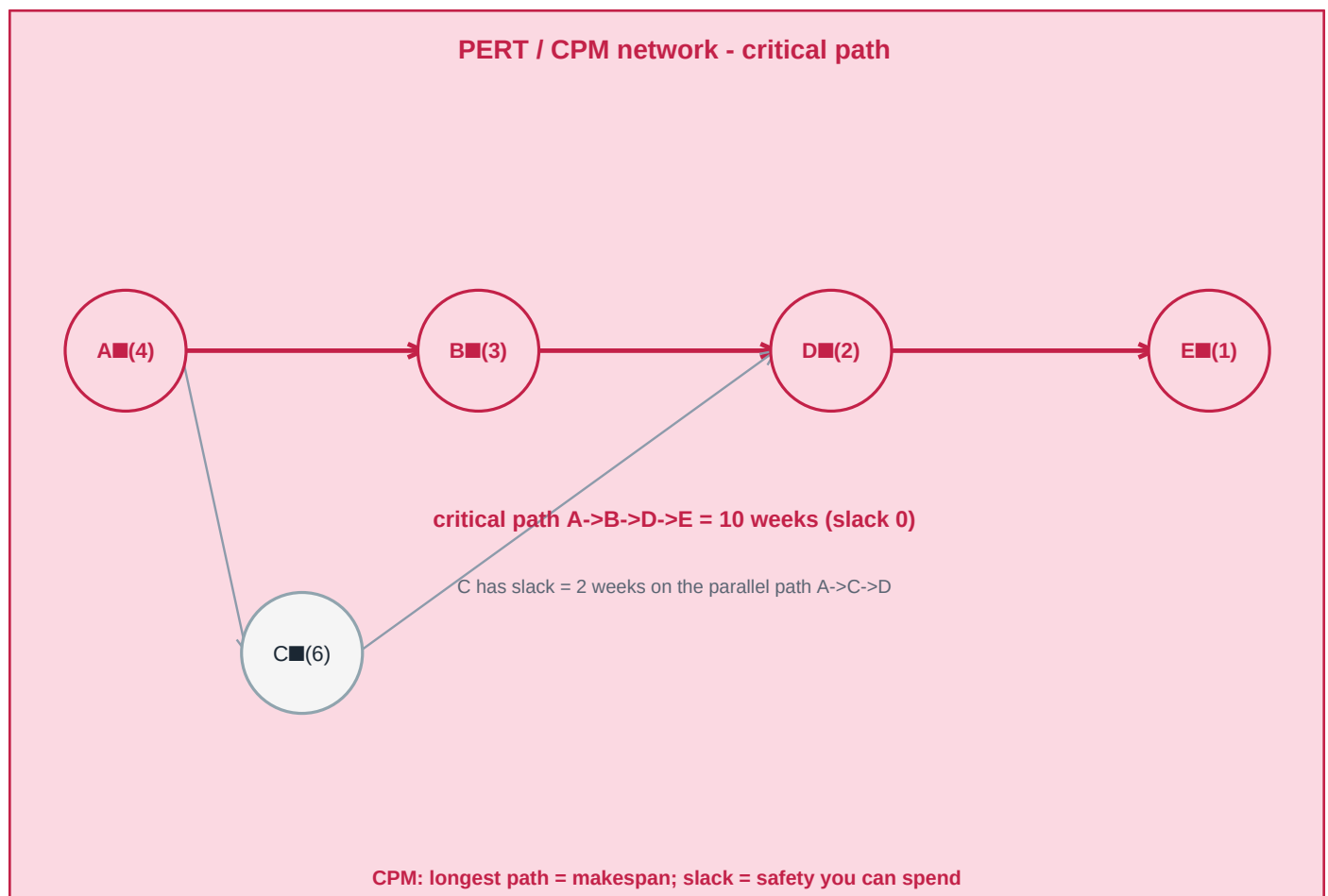
Effort $E = a(KLOC)^b$ person-months; development time $D = c(E)^d$ months. Organic mode ($a=2.4, b=1.05$, well-understood small proj), Semi-detached ($a=3.0, b=1.12$), Embedded ($a=3.6, b=1.20$, tight hardware/real-time constraints). Basic COCOMO is the quick first-pass; intermediate adds 15 cost-driver multipliers (product, platform, personnel, project).

COCOMO.java.txt

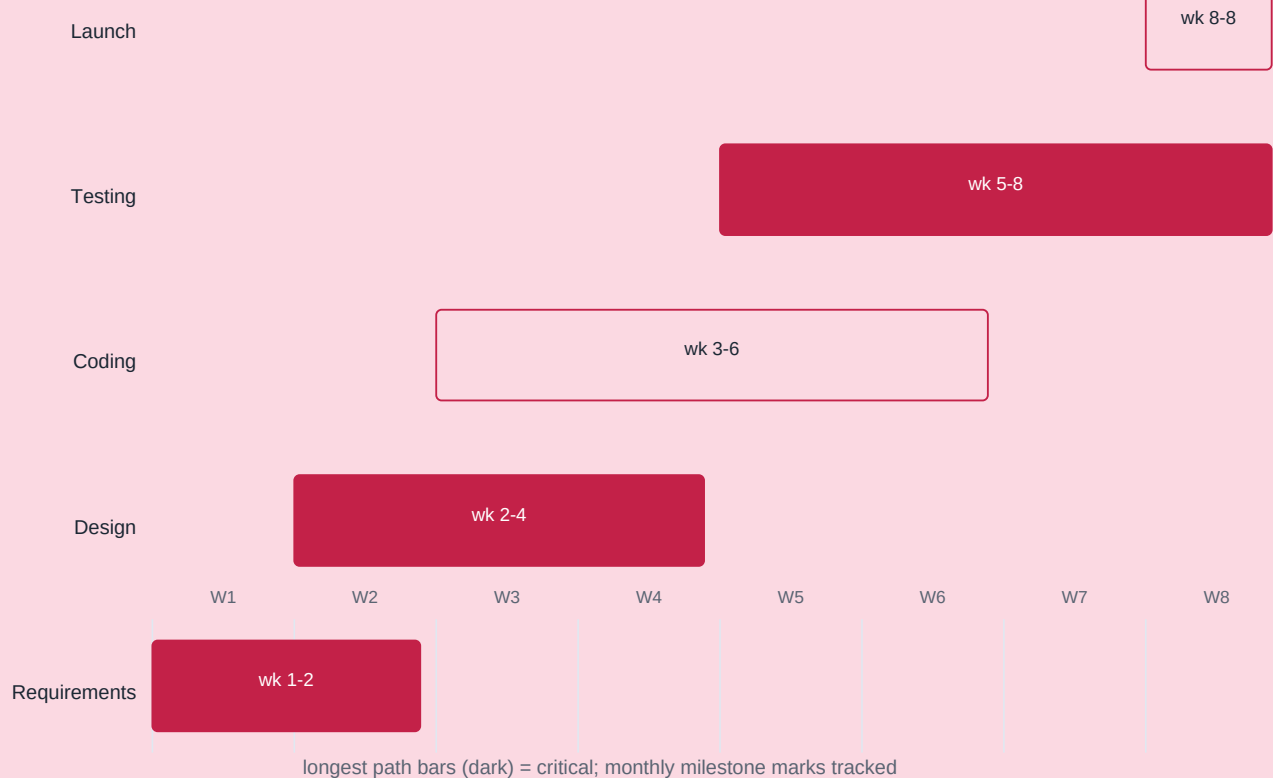
```
// basic COCOMO example - a 30 KLOC organically-derived module:
// E = 2.4 * (30)^1.05 person-months (organic)
double E = 2.4 * Math.pow(30, 1.05); // ~ 82 person-months
double D = 2.5 * Math.pow(E, 0.38); // -> ~ 12.5 months dev time
double staff = Math.ceil(E / D); // ~ 7 developers
// compare with embedded: E = 3.6*(30)^1.20 -> steeper effort & more risk
```

5.6 Scheduling - Gantt, PERT/CPM & Tracking**WHY SCHEDULE**

A schedule orders tasks, allocates resources and defines checkpoints so we can TRACK progress against plan. Tools: WBS (divide into tasks), activity networks (dependencies), Gantt bars, PERT/CPM critical-path analysis.



Gantt Chart - 8 week project schedule



Gantt = visual schedule: bars show when tasks run (weeks)

CRITICAL PATH METHOD (CPM / PERT)

Draw the activity-on-node network: nodes = events, edges = tasks with durations. Compute EARLIEST start (forward pass) and LATEST start (backward pass). The CRITICAL PATH = longest path; SLACK/float of a task = how long it can slip without delaying the project. Manage the critical path (it is the true schedule); PERT adds three-point estimates (optimistic/expected/pessimistic) for uncertainty.

CPM CRITICAL PATH + SLACK EXAMPLE

```

A(4) --> B(3) --> D(2) --> E(1)    critical path A-B-D-E = 10 wks (slack 0)
|         |         |                parallel task C(6) branches from A..E
+-----> C(6) -----+                slack(C) = (start D) - (end C) = 8-6 = 2 wks
tract: compare actual %complete vs planned -> SPI; milestone review monthly
  
```

5.7 Risk Management & SQA

RISK MANAGEMENT

Identify the events that could hurt the project (schedule slip, staff loss, requirement churn, tech failure), analyse likelihood x impact, then plan: AVOID (remove cause), MITIGATE (reduce likelihood/impact), CONTINGENCY (backup plan), or ACCEPT. Track the top-10 risks each week.



Risk (example)	Likelihood	Impact	Mitigation
Key developer leaves	M	High	pair knowledge, docs, backups
Requirements creep	H	High	CCB, time-boxing, baseline SRS
Schedule slip	M	M	buffer, milestone reviews, re-plan
COTS vendor drops	L	H	in-house fallback, license EULA

SOFTWARE QUALITY ASSURANCE (SQA)

SQA = the planned, systematic pattern of actions to CONFIRM quality: (1) standards (ISO 9001, CMMI), (2) reviews & audits at each phase, (3) formal SQA plan (quality goals, metrics, verification), (4) testing/v&v and defect statistics feeding the process. QA differs from QC/Testing: QA prevents defects, testing detects them.

PROJECT METRICS RETURN (MEASUREMENT RECAP)

Size (LOC/FP) feeds estimation; effort & schedule slip show planning quality; defect density and DRE show testing quality; maintenance per type shows product health. "You cannot control what you do not measure" - each metric must be tied to a decision, not collected for show.

Exercise: 1) Classify 4 recent "change requests" on a phone app into maintenance types. 2) Trace git add/commit/tag for a 3-baseline product. 3) Estimate FP for a bus booking (6 inputs, 3 outputs, 2 files) and convert to LOC (Java). 4) Find the critical path + slack for tasks A(3), B(4), C(2), D(5) where B & C depend on A, D depends on B & C. 5) List top-5 risks for your own group project with a mitigation each.